

DATA ENCODINGS

- > ONE HOT ENCODING
- > ORDINAL ENCODING
- > LABEL ENCODING



MACHINE LEARNING

❏ Encoding Techniques in Machine Learning:

Encoding techniques in machine learning refer to the methods used to transform categorical data into numerical data, allowing algorithms to process and analyze it effectively. Since most machine learning models work with numerical inputs, encoding is a critical preprocessing step for handling categorical variables.

1. One-Hot Encoding (OHE)

Definition:

One-Hot Encoding is a technique that converts categorical variables into a binary (0 or 1) representation. Each category is transformed into a new binary feature (column), and only one of these features is "hot" (1) for each observation, while the rest are "cold" (0).

If a feature has n unique categories, OHE creates n new binary features.

Example:

Suppose you have a categorical feature **Color** with three categories: Red, Green, and Blue.

Original Data:

Color
Red
Green
Blue
Red

After One-Hot Encoding:

Color_Red	Color_Green	Color_Blue
1	0	0
0	1	0
0	0	1

❑ When to Use One-Hot Encoding?

- When categorical values are **nominal** (no inherent order or ranking) like color, gender, city names.
- When the number of unique categories is small to moderate (to prevent memory issues or to avoid the "curse of dimensionality").
- When the categorical variable is not ordinal, so numerical relationships do not exist.
- When the machine learning algorithm cannot handle categorical data directly (e.g., linear regression, neural networks).

Advantages

- ✓ Prevents the model from assuming an ordinal relationship between categories.
- ✓ Works well with algorithms that do not assume order, such as decision trees and neural networks.

Disadvantages

- ✗ Increases dimensionality significantly when dealing with high-cardinality categorical features.
- ✗ Can lead to the "curse of dimensionality" if applied to large datasets with many unique categories.

2. Label Encoding

Definition:

Label Encoding assigns a unique integer to each category in a categorical variable. The integers are arbitrary and do not imply any order or ranking.

Color

Red

Green

Blue

Red

After Label Encoding:

Color

0

1

2

0

❑ When to Use Label Encoding?

- When the categorical variable is **nominal** and the algorithm can handle numerical input (e.g., tree-based models like decision trees, random forests).
- Use when dealing with categorical data where the order **does not matter** but the dataset has a limited number of unique categories (e.g., binary or simple categorical data).
 - Example: Gender ("Male", "Female"), Fruit types ("Apple", "Banana", "Cherry").
- When the number of unique categories is large, and One-Hot Encoding would lead to high dimensionality.

Advantages

- ✓ Reduces memory usage and improves computation efficiency compared to One-Hot Encoding.
- ✓ Works well with models that can handle numerical relationships correctly.

Disadvantages

- ✗ Introduces an artificial ordinal relationship between categories, which can mislead linear models.
- ✗ Can create bias if the encoded values imply a ranking that does not actually exist.

3. Ordinal Encoding

Definition

Ordinal Encoding is similar to Label Encoding but is specifically used when the categorical feature has an inherent order. Each category is assigned a numerical value based on its rank.

Original Data (Education Levels):

Education Level
High School
Bachelor's
Master's
PhD

Ordinal Encoded Representation:

Education Level (Encoded)
0
1
2
3

Rating	Ordinal Encoded Value
Bad	0
Good	1
Better	2

Explanation:

- Since "**Bad**" < "**Good**" < "**Better**", we assign increasing numerical values accordingly.

❏ When to Use Ordinal Encoding?

- When categorical values have a clear, meaningful ranking or order (e.g., education level, salary grades).
- When preserving the order of categories is important for model learning.
- Works well with both linear models and tree-based models when order matters.

Advantages

- ✓ Retains meaningful order information, making it useful for models that benefit from ordinal relationships.
- ✓ More memory-efficient than One-Hot Encoding.

Disadvantages

- ✗ If used on non-ordinal data, it introduces an incorrect relationship between categories.
- ✗ Can mislead linear models if the ordinal values do not represent actual equidistant differences.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, OrdinalEncoder
```

```
# Create a sample dataframe
```

```
data = {
    'Color': ['Red', 'Blue', 'Green', 'Red', 'Blue', 'Green', 'Red', 'Green'],
    'Size': ['Small', 'Medium', 'Large', 'Medium', 'Large', 'Small', 'Large', 'Small'],
    'Target': [1, 0, 1, 1, 0, 0, 1, 1]
}
df = pd.DataFrame(data)
```

df

	Color	Size	Target
0	Red	Small	1
1	Blue	Medium	0
2	Green	Large	1
3	Red	Medium	1
4	Blue	Large	0
5	Green	Small	0
6	Red	Large	1
7	Green	Small	1

```
# Label Encoding  
le = LabelEncoder()  
df['Color_LabelEncoded'] = le.fit_transform(df['Color'])
```

df

	Color	Size	Target	Color_LabelEncoded
0	Red	Small	1	2
1	Blue	Medium	0	0
2	Green	Large	1	1
3	Red	Medium	1	2
4	Blue	Large	0	0
5	Green	Small	0	1
6	Red	Large	1	2
7	Green	Small	1	1

```
# Ordinal Encoding
```

```
ord_enc = OrdinalEncoder(categories=[['Small', 'Medium', 'Large']])  
df['Size_OrdinalEncoded'] = ord_enc.fit_transform(df[['Size']])
```

df

	Color	Size	Target	Color_LabelEncoded	Color_Blue	Color_Green	Color_Red	Size_OrdinalEncoded
0	Red	Small	1	2	0.0	0.0	1.0	0.0
1	Blue	Medium	0	0	1.0	0.0	0.0	1.0
2	Green	Large	1	1	0.0	1.0	0.0	2.0
3	Red	Medium	1	2	0.0	0.0	1.0	1.0
4	Blue	Large	0	0	1.0	0.0	0.0	2.0
5	Green	Small	0	1	0.0	1.0	0.0	0.0
6	Red	Large	1	2	0.0	0.0	1.0	2.0
7	Green	Small	1	1	0.0	1.0	0.0	0.0

❑ Ordinal Encoding (Using map()):

```
# Sample Data
```

```
data = pd.DataFrame({'Education': ['High School', 'Bachelor', 'Master', 'PhD', 'Bachelor']})  
data
```

Education	
0	High School
1	Bachelor
2	Master
3	PhD
4	Bachelor

```
# Define ordinal ranking
```

```
education_mapping = {'High School': 0, 'Bachelor': 1, 'Master': 2, 'PhD': 3}  
data['Education_OrdinalEncoded'] = data['Education'].map(education_mapping)
```

```
# Display the result
```

```
print(data)
```

	Education	Education_OrdinalEncoded
0	High School	0
1	Bachelor	1
2	Master	2
3	PhD	3
4	Bachelor	1

❑ One-Hot Encoding using OneHotEncoder from Scikit-Learn:

```
# One-Hot Encoding
ohe = OneHotEncoder(sparse=False)
ohe_df = pd.DataFrame(ohe.fit_transform(df[['color']]), columns=ohe.get_feature_names_out(['color']))
df = pd.concat([df, ohe_df], axis=1)
```

df

	Color	Size	Target	Color_LabelEncoded	Color_Blue	Color_Green	Color_Red
0	Red	Small	1	2	0.0	0.0	1.0
1	Blue	Medium	0	0	1.0	0.0	0.0
2	Green	Large	1	1	0.0	1.0	0.0
3	Red	Medium	1	2	0.0	0.0	1.0
4	Blue	Large	0	0	1.0	0.0	0.0
5	Green	Small	0	1	0.0	1.0	0.0
6	Red	Large	1	2	0.0	0.0	1.0
7	Green	Small	1	1	0.0	1.0	0.0

Now supported,
sparse_output=False

❑ One-Hot Encoding using pandas.get_dummies():

pandas.get_dummies() is a function used to perform **One-Hot Encoding** on categorical data by converting each unique category into a new binary (0 or 1) column. This is useful for converting **nominal categorical variables** into a format suitable for machine learning models.

```
import pandas as pd
# Sample Data
data = pd.DataFrame({'Vehicle': ['Car', 'Bike', 'Truck', 'Bike', 'Car']})
data
```

Vehicle	
0	Car
1	Bike
2	Truck
3	Bike
4	Car

```
# Apply One-Hot Encoding with drop_first=True
data_one_hot = pd.get_dummies(data, columns=['Vehicle'], prefix='Vehicle', drop_first=True)

# Display the result
print(data_one_hot)
```

Output

Vehicle_Bike	Vehicle_Truck
0	0
1	0
0	1
1	0
0	0

Conclusion: When to Use One-Hot Encoding, Ordinal Encoding, and Label Encoding?

Encoding Type	When to Use?	Suitable Data Types	Output Type
One-Hot Encoding	When categories do not have any ordinal relationship (e.g., Colors, Countries, Cities).	Categorical (Nominal)	Multiple binary columns (0s & 1s).
Ordinal Encoding	When categories have a meaningful order/rank (e.g., Low < Medium < High, Bad < Better < Good).	Categorical (Ordinal)	Single numerical column with ranked values.
Label Encoding	When categories are unordered but need numeric conversion (mainly for algorithms that don't support categorical data, e.g., Decision Trees, XGBoost).  May introduce an unintended order!	Categorical (Nominal & Ordinal)	Single numerical column with assigned labels.